Docker — 3–4 YOE Scenario-Based Q&A

1. Docker container starts and exits immediately. What will you check?
Ans: First, I check the container status and Docker logs to understand why the container stopped. I look for application errors, configuration issues, or any error message that explains why the process exited.
Then I check the CMD and ENTRYPOINT configured in the Dockerfile and verify whether the application process is starting correctly. I also check the container exit code to understand the reason for termination.
Simple ga gurthupettuko: Container start ayyi exit ayithe → first status and logs chudu → application enduku stop ayyindo identify cheyyi → CMD and ENTRYPOINT check cheyyi → application process correct ga start avutunda chudu → exit code verify cheyyi → issue fix chesi container malli run cheyyi.
Eng: If the container starts and exits → first check the status and logs → identify why the application stopped → check CMD and ENTRYPOINT → verify the application process → check the exit code → fix the issue and run the container again.
Example:
docker ps -a → Container status and exit status chudu.
docker logs <container_id> → Container enduku stop ayyindo error chudu.
docker inspect <container_id> --format='{{.State.ExitCode}}' → Container exit code ento check cheyyi.
docker inspect <container_id> → CMD and ENTRYPOINT configuration verify cheyyi.
Issue fix chesi → docker start <container_id> → Container malli successfully run avutunda verify cheyyi.

---

2. Docker image build is failing. What will you check?
Ans: First, I check the Docker build output and identify the exact step where the build is failing. I look at the error message to understand whether the issue is related to the Dockerfile, package installation, or application files.
Then I verify the base image and check whether it is available and compatible with the application. I also check whether the required application files and dependencies are being copied and installed correctly.
Simple ga gurthupettuko: Image build fail ayithe → first error ekkada vachindo identify cheyyi → aa Dockerfile step ni check cheyyi → base image correct ga unda chudu → dependencies and package installation verify cheyyi → network issue unda check cheyyi → issue fix chesi image ni rebuild cheyyi.
Eng: If the image build fails → first identify the error location → check the failed Dockerfile step → verify the base image → check dependencies and package installation → verify network connectivity → fix the issue and rebuild the image.
Example:
docker build -t myapp:latest . → Image build start chesi fail avutunda chudu.
docker build -t myapp:latest . → Output lo exact error ekkada vachindo chudu.
cat package.json → Required dependencies unnaya check cheyyi.
ping registry.npmjs.org → Network connectivity unda check cheyyi.
npm install → Dependencies install avutunnaya, exact error em vastundo chudu.
Issue fix chesi → docker build -t myapp:latest . → Image build successful ga ayyinda verify cheyyi.

---

3. Docker image is very large. How will you reduce the image size?
Ans: First, I check the Docker image layers and identify which layers or files are consuming more space. I review the Dockerfile and look for unnecessary packages, files, and build dependencies.
Then I use a smaller base image when it is suitable for the application. I also use multi-stage builds so that build-time dependencies and unnecessary files are not included in the final production image.
Simple ga gurthupettuko: Image size ekkuva unte → first ye layers ekkuva space tiskuntunnayo chudu → unnecessary packages and files identify cheyyi → suitable smaller base image use cheyyi → build dependencies final image lo raakunda chudu → multi-stage build use cheyyi → final image size verify cheyyi.
Eng: If the image size is too large → first identify the layers using more space → check unnecessary packages and files → use a smaller suitable base image → keep build dependencies out of the final image → use a multi-stage build → verify the final image size.
Example:
docker images myapp → Current image size entha undo chudu.
docker history myapp:latest → Ye image layers ekkuva size tiskuntunnayo check cheyyi.
cat Dockerfile → Unnecessary packages and build steps unnaya chudu.
cat .dockerignore → Unnecessary files build context loki vellakunda unnaya check cheyyi.
Dockerfile lo suitable smaller base image and multi-stage build use chesi → docker build -t myapp:latest . → Final image size malli verify cheyyi.

---

4. Docker container works locally but not in production. What will you check?
Ans: First, I compare the local and production environments to identify what is different. I check the Docker image version, environment variables, application configuration, ports, volumes, and secrets.
Then I check the application logs and network connectivity in production. I also verify that the same Docker image is being deployed and that all required dependencies and configuration values are available in production.
Simple ga gurthupettuko: Local lo work ayyi Prod lo fail ayithe → first local and Prod difference chudu → image version and configuration check cheyyi → variables, ports, volumes, secrets verify cheyyi → production logs and network chudu → same image deploy ayyinda check cheyyi → required dependencies verify cheyyi.
Eng: If the container works locally but fails in production → first compare both environments → check image version and configuration → verify variables, ports, volumes, and secrets → check production logs and network → confirm the same image is deployed → verify dependencies.
Example:
docker image inspect myapp:latest → Deploy chesthunna image details verify cheyyi.
docker inspect <container_id> → Production container environment variables, ports, and volumes check cheyyi.
docker logs <container_id> → Production application error em vastundo chudu.

docker exec -it <container_id> env → Required environment variables available unnaya check cheyyi.
docker exec -it <container_id> curl http://localhost:<port> → Application container lopala accessible ga unda check cheyyi.
Difference identify chesi → configuration or image issue fix chesi → container malli run chesi → production application verify cheyyi.

---

5. Docker container is running, but the application is not accessible. What will you check?
Ans: First, I check whether the application is listening on the expected port inside the container. Then I verify whether the Docker port mapping between the host and container is configured correctly.
I also check the application logs and network configuration to make sure the application started successfully and is accepting connections on the expected interface and port.
Simple ga gurthupettuko: Container running unna app access avvakapothe → first application ye port lo listen chestundo chudu → Docker port mapping check cheyyi → application logs verify cheyyi → network configuration chudu → expected interface and port correct ga unnaya check cheyyi.
Eng: If the container is running but the application is not accessible → first check the application port → verify Docker port mapping → check application logs → verify network configuration → confirm the expected interface and port.
Example:
docker ps → Container running lo unda and port mapping em undo chudu.
docker port <container_id> → Host port and container port mapping verify cheyyi.
docker logs <container_id> → Application successfully start ayyinda check cheyyi.
docker exec -it <container_id> ss -lntp → Container lopala application ye port lo listen chesthundo chudu.
curl http://localhost:<host_port> → Host nunchi application accessible ga unda test cheyyi.
Port or application configuration fix chesi → curl http://localhost:<host_port> → Application access avutunda verify cheyyi.

---

6. One Docker container cannot connect to another container. What will you check?
Ans: First, I check whether both containers are running and connected to the expected Docker network. Then I verify whether the application is using the correct container name and port for communication.
I also check the Docker network configuration and DNS resolution between the containers. Finally, I review the application logs to identify whether the problem is with networking or the application itself.
Simple ga gurthupettuko: One container nunchi another ki connect avvakapothe → first rendu containers running unnaya chudu → same Docker network lo unnaya check cheyyi → container name and port verify cheyyi → DNS resolution chudu → network issue aa application issue aa identify cheyyi.
Eng: If one container cannot connect to another → first check both containers → verify the Docker network → check the container name and port → verify DNS resolution → identify whether the issue is network or application related.
Example:
docker ps → Rendu containers running unnaya chudu.
docker network ls → Required Docker network unda check cheyyi.
docker network inspect <network_name> → Rendu containers same network lo unnaya verify cheyyi.
docker exec -it <container_id> ping <target_container_name> → Container name resolve avutunda and connectivity unda test cheyyi.
docker exec -it <container_id> curl http://<target_container_name>:<port> → Target application port reachable ga unda check cheyyi.
Network or port issue fix chesi → curl http://<target_container_name>:<port> → Container-to-container communication verify cheyyi.

---

7. Container data is lost after the container is removed. What will you do?
Ans: First, I check whether the application data is being stored inside the container's writable filesystem. Data stored only inside the container can be lost when the container is removed.
Then I use Docker volumes or an external storage solution for data that must survive container deletion. I also verify that the volume is mounted to the correct application path.
Simple ga gurthupettuko: Container remove chesaka data pothunte → first data ekkada store avutundo chudu → container filesystem lo store chesthunnara check cheyyi → persistent data kosam Docker volume use cheyyi → correct application path ki mount ayyinda verify cheyyi → data retain avutunda check cheyyi.
Eng: If data is lost after removing the container → first check where the data is stored → verify whether it is inside the container filesystem → use a Docker volume for persistent data → verify the correct mount path → confirm the data is retained.
Example:
docker inspect <container_id> → Container ki volume mount ayyinda chudu.
docker volume ls → Docker volume create ayyinda check cheyyi.
docker volume inspect <volume_name> → Volume details and mount location verify cheyyi.
docker run -v mydata:/app/data myapp:latest → Application data kosam persistent volume attach cheyyi.
Container remove chesi → same volume tho new container run chesi → data still available ga unda verify cheyyi.

---

8. Docker image push to the registry is failing. What will you check?
Ans: First, I check whether the image is tagged correctly with the registry address, repository name, and required tag. Then I verify whether Docker is authenticated with the registry.
I also check whether the user has permission to push to the repository and whether network connectivity to the registry is working. After fixing the issue, I try the push again.
Simple ga gurthupettuko: Image push fail ayithe → first image tag correct ga unda chudu → registry login verify cheyyi → repository push permission check cheyyi → registry network connectivity chudu → issue fix chesi image ni push malli run cheyyi.
Eng: If the image push fails → first check the image tag → verify registry login → check repository push permission → verify registry connectivity → fix the issue and push the image again.
Example:
docker images → Image name and tag correct ga unnaya chudu.

docker tag myapp:latest <registry>/myapp:latest → Correct registry repository tag create cheyyi.
docker login <registry> → Registry authentication successful ga avutunda check cheyyi.
docker push <registry>/myapp:latest → Image push chesthe exact error em vastundo chudu.
Permission or authentication issue fix chesi → docker push <registry>/myapp:latest → Push successful ga ayyinda verify cheyyi.

---

9. Docker image pull is failing. What will you check?
Ans: First, I check whether the image name, repository, and tag are correct. Then I verify whether the requested image actually exists in the Docker registry.
I also check registry authentication, permissions, and network connectivity. If the repository is private, I verify that the system pulling the image has the required access.
Simple ga gurthupettuko: Image pull fail ayithe → first image name, repository, tag correct aa chudu → registry lo image unda verify cheyyi → authentication and permissions check cheyyi → network connectivity chudu → private repository ayithe required access unda confirm cheyyi.
Eng: If the image pull fails → first check the image name, repository, and tag → verify the image exists → check authentication and permissions → verify network connectivity → confirm access for a private repository.
Example:
docker pull <registry>/myapp:latest → Image pull chesi exact error em vastundo chudu.
docker login <registry> → Private registry authentication check cheyyi.
docker images → Image local system lo pull ayyinda verify cheyyi.
ping <registry> → Registry network connectivity unda check cheyyi.
Image name, permission, or network issue fix chesi → docker pull <registry>/myapp:latest → Pull successful ga ayyinda verify cheyyi.

---

10. Docker container is consuming high CPU. What will you check?
Ans: First, I check the container CPU usage and identify whether the application or another process is consuming excessive CPU. Then I check the application logs and running processes inside the container.
I also review the configured CPU limits and investigate the application behavior before increasing resources. The main goal is to identify and fix the root cause instead of only increasing CPU.
Simple ga gurthupettuko: Container CPU ekkuva use chesthunte → first CPU usage chudu → ye process ekkuva CPU use chestundo identify cheyyi → application logs check cheyyi → CPU limits verify cheyyi → application behavior chudu → root cause fix cheyyi → required ayithe resources increase cheyyi.
Eng: If the container is using high CPU → first check CPU usage → identify the high-CPU process → check application logs → verify CPU limits → investigate application behavior → fix the root cause → increase resources if required.
Example:
docker stats <container_id> → Container CPU usage entha undo chudu.
docker exec -it <container_id> top → Container lopala ye process CPU ekkuva use chesthundo identify cheyyi.
docker logs <container_id> → Application lo repeated errors or heavy processing unda chudu.
docker inspect <container_id> → CPU limits configured unnaya verify cheyyi.
Root cause fix chesi → docker stats <container_id> → CPU usage normal ki vachinda verify cheyyi.

---

11. Docker container is running out of memory. What will you check?
Ans: First, I check the container memory usage and logs to understand when and why the memory usage is increasing. Then I verify whether the application is consuming more memory than expected or has a possible memory leak.
I also check the configured memory limit and determine whether the limit is appropriate for the application. After identifying the root cause, I make the required change and verify the container again.
Simple ga gurthupettuko: Container memory aipothe → first memory usage and logs chudu → memory enduku increase avutundo identify cheyyi → application ekkuva memory use chesthunda check cheyyi → memory leak unda verify cheyyi → memory limit chudu → root cause fix chesi container verify cheyyi.
Eng: If the container runs out of memory → first check memory usage and logs → identify why memory is increasing → check excessive application memory usage → verify a possible memory leak → check the memory limit → fix the root cause and verify the container.
Example:
docker stats <container_id> → Container memory usage entha undo chudu.
docker logs <container_id> → Memory-related application errors unnaya check cheyyi.
docker exec -it <container_id> top → Ye process ekkuva memory use chesthundo identify cheyyi.
docker inspect <container_id> → Memory limit configured unda verify cheyyi.
Memory issue fix chesi → docker stats <container_id> → Memory usage normal ga unda verify cheyyi.

---

12. Dockerfile has many unnecessary layers. How will you optimize it?
Ans: First, I review the Dockerfile and identify unnecessary commands, packages, and files that are creating extra layers or increasing the image size.
Then I optimize the Dockerfile by removing unnecessary steps and using suitable commands and base images. I also use .dockerignore to prevent unnecessary files from being copied into the build context and use multi-stage builds when required.
Simple ga gurthupettuko: Dockerfile lo layers ekkuva unte → first unnecessary commands and packages identify cheyyi → unwanted files check cheyyi → Dockerfile steps optimize cheyyi → suitable base image use cheyyi → .dockerignore tho unnecessary files avoid cheyyi → required ayithe multi-stage build use cheyyi.
Eng: If the Dockerfile has too many layers → first identify unnecessary commands and packages → check unwanted files → optimize Dockerfile steps → use a suitable base image → avoid unnecessary files with .dockerignore → use a multi-stage build when required.
Example:
cat Dockerfile → Dockerfile lo unnecessary RUN, COPY, and package commands unnaya chudu.
cat .dockerignore → Unwanted files build context loki vellakunda unnaya check cheyyi.
docker history myapp:latest → Existing image lo unnecessary layers identify cheyyi.

Dockerfile optimize chesi → docker build -t myapp:latest . → New image build chesi size and layers verify cheyyi.

---

13. Docker image build is taking too much time. What will you check?
Ans: First, I identify which Dockerfile instruction is taking the most time by checking the build output. Then I review the Dockerfile instruction order because Docker layer caching can reduce build time.
I also check whether unnecessary files are being copied into the build context and whether dependencies are being downloaded repeatedly. After identifying the slow step, I optimize the Dockerfile and build process.
Simple ga gurthupettuko: Build slow ga unte → first ye Dockerfile step slow ga undho chudu → layer cache work avutunda check cheyyi → instruction order verify cheyyi → unnecessary files copy avutunnaya chudu → dependencies repeated ga download avutunnaya check cheyyi → build process optimize cheyyi.
Eng: If the build is slow → first identify the slow Dockerfile step → check layer caching → verify instruction order → check unnecessary files → verify repeated dependency downloads → optimize the build process.
Example:
docker build -t myapp:latest . → Build output lo ye step ekkuva time tiskuntundo chudu.
cat Dockerfile → Dockerfile instruction order correct ga unda check cheyyi.
cat .dockerignore → Unnecessary files build context loki copy avutunnaya chudu.
docker build --no-cache -t myapp:test . → Cache lekunda build chesi actual slow step identify cheyyi.
Dockerfile and cache usage optimize chesi → docker build -t myapp:latest . → Build time improve ayyinda verify cheyyi.

---

14. Docker container is restarting continuously. What will you check?
Ans: First, I check the container status and logs to identify why the application process is stopping. Then I check the container exit code and startup configuration.
I also verify the health check and restart policy because an unhealthy or repeatedly failing application can cause the container to restart continuously. After identifying the cause, I fix the issue and verify the container behavior.
Simple ga gurthupettuko: Container repeated ga restart avutunte → first status and logs chudu → application enduku stop avutundo identify cheyyi → exit code check cheyyi → startup configuration verify cheyyi → health check and restart policy chudu → issue fix chesi container behavior verify cheyyi.
Eng: If the container keeps restarting → first check status and logs → identify why the application stops → check the exit code → verify startup configuration → check health check and restart policy → fix the issue and verify the container.
Example:
docker ps -a → Container repeated ga restart avutunda chudu.
docker logs <container_id> → Application enduku stop avutundo error chudu.
docker inspect <container_id> --format='{{.State.ExitCode}}' → Exit code verify cheyyi.
docker inspect <container_id> → Restart policy and health check configuration chudu.
Issue fix chesi → docker restart <container_id> → Container stable ga run avutunda verify cheyyi.

---

15. Docker container cannot access the internet. What will you check?
Ans: First, I check whether the container has network connectivity and whether the host itself can access the internet. Then I verify the Docker network and DNS configuration.
I also check proxy settings, firewall rules, and security restrictions that may be blocking outbound traffic from the container. After identifying the cause, I correct the network configuration and test the connection again.
Simple ga gurthupettuko: Container internet access avvakapothe → first container network chudu → host internet work avutunda check cheyyi → Docker network and DNS verify cheyyi → proxy settings check cheyyi → firewall and security rules chudu → issue fix chesi connection malli test cheyyi.
Eng: If the container cannot access the internet → first check container networking → verify host internet access → check Docker network and DNS → verify proxy settings → check firewall and security rules → fix the issue and test the connection again.
Example:
ping google.com → Host machine ki internet connectivity unda check cheyyi.
docker exec -it <container_id> ping google.com → Container nunchi internet reach avutunda test cheyyi.
docker exec -it <container_id> cat /etc/resolv.conf → Container DNS configuration chudu.
docker network inspect bridge → Docker network configuration verify cheyyi.
DNS, proxy, or firewall issue fix chesi → docker exec -it <container_id> ping google.com → Internet connectivity malli verify cheyyi.

---

16. Docker container is not starting because of configuration issues. What will you check?
Ans: First, I check the environment variables and configuration values passed to the container. Then I compare them with the values required by the application.
I also verify secrets, configuration files, and the CMD or ENTRYPOINT used to start the application. After correcting the configuration, I start the container again and check the logs to confirm that the application starts successfully.
Simple ga gurthupettuko: Configuration valla container start avvakapothe → first environment variables check cheyyi → application required values tho compare cheyyi → secrets and config files verify cheyyi → CMD and ENTRYPOINT chudu → configuration fix chesi container malli start cheyyi → logs verify cheyyi.
Eng: If the container does not start because of configuration → first check environment variables → compare them with application requirements → verify secrets and config files → check CMD and ENTRYPOINT → fix the configuration → start the container again → verify the logs.
Example:
docker run -e APP_ENV=prod myapp:latest → Required environment variable pass chesthe container start avutunda chudu.
docker inspect <container_id> → Environment variables and startup configuration verify cheyyi.
docker logs <container_id> → Configuration-related error exact ga em vastundo chudu.
docker exec -it <container_id> env → Container lopala required variables available unnaya check cheyyi.
Configuration fix chesi → container malli start chesi → docker logs <container_id> → Application successfully start ayyinda verify cheyyi.

17. Docker image is built successfully, but the application is not working. What will you do?
Ans: First, I start the container and check the application logs to understand the runtime issue. A successful image build only confirms that the image was created; it does not guarantee that the application will run correctly.
Then I verify environment variables, dependencies, ports, volumes, and the startup command. I compare the container environment with the environment where the application was working correctly and fix the difference.
Simple ga gurthupettuko: Image build successful ayina app work avvakapothe → first container start chesi logs chudu → runtime issue identify cheyyi → variables and dependencies verify cheyyi → ports and volumes check cheyyi → startup command verify cheyyi → working environment tho compare chesi difference fix cheyyi.
Eng: If the image builds successfully but the application does not work → first start the container and check logs → identify the runtime issue → verify variables and dependencies → check ports and volumes → verify the startup command → compare the working environment and fix the difference.
Example:
docker run -d --name myapp myapp:latest → Image nunchi container start cheyyi.
docker logs myapp → Application runtime error em vastundo chudu.
docker inspect myapp → Environment variables, ports, and volumes verify cheyyi.
docker exec -it myapp env → Required environment variables available unnaya check cheyyi.
docker exec -it myapp <application-command> → Application process correct ga start avutunda check cheyyi.
Issue fix chesi → docker restart myapp → Application working ga unda verify cheyyi.

---

18. How will you troubleshoot a Docker production issue?
Ans: First, I check the container status, application logs, and recent changes to understand what caused the production issue. I also check whether a new image version or configuration change was recently deployed.
Then I verify CPU, memory, network, ports, volumes, environment variables, and secrets. After identifying the root cause, I make the required change, test the application, and verify that the production service is working correctly.
Simple ga gurthupettuko: Docker Prod issue vaste → first status and logs chudu → recent changes and image version check cheyyi → CPU and memory verify cheyyi → network, ports, and volumes check cheyyi → variables and secrets chudu → root cause fix chesi application test cheyyi → Prod service verify cheyyi.
Eng: If there is a Docker production issue → first check status and logs → review recent changes and image version → verify CPU and memory → check network, ports, and volumes → verify variables and secrets → fix the root cause → test the application and verify the production service.
Example:
docker ps → Production containers running lo unnaya chudu.
docker logs <container_id> → Production application error em vastundo check cheyyi.
docker stats <container_id> → CPU and memory usage normal ga unnaya verify cheyyi.
docker inspect <container_id> → Ports, volumes, and environment configuration check cheyyi.
docker image inspect <image> → Recently deployed image version verify cheyyi.
Issue identify chesi → required fix apply chesi → application test chesi → docker ps and logs tho production service verify cheyyi.

---

19. Old Docker images are consuming a lot of disk space. What will you do?
Ans: First, I identify unused images, stopped containers, unused volumes, and build cache that are consuming disk space. Then I determine which resources are safe to remove.
I follow the environment cleanup policy and make sure that images, containers, or volumes required by running production applications are not deleted. After cleanup, I verify the available disk space.
Simple ga gurthupettuko: Docker disk space ekkuva use chesthunte → first unused images and stopped containers identify cheyyi → unused volumes and build cache check cheyyi → safe ga remove cheyyagalige resources verify cheyyi → Prod resources delete cheyyakunda cleanup cheyyi → disk space malli verify cheyyi.
Eng: If Docker is consuming too much disk space → first identify unused images and stopped containers → check unused volumes and build cache → verify what can be safely removed → clean up without deleting production resources → verify the disk space again.
Example:
docker system df → Docker disk space ekkuva ekkada use avutundo chudu.
docker images → Old and unused images identify cheyyi.
docker ps -a → Stopped containers unnaya check cheyyi.
docker volume ls → Unused volumes unnaya chudu.
docker builder prune → Unused build cache cleanup cheyyi.
Safe resources identify chesi → cleanup perform chesi → docker system df → Disk space improve ayyinda verify cheyyi.

---

20. How will you securely handle secrets in Docker?
Ans: First, I make sure passwords, API keys, and other sensitive values are not hardcoded in the Dockerfile or source code. Secrets should not become part of the Docker image.
Then I use a secure secret-management mechanism and provide secrets at runtime according to the environment. I also make sure secrets are not exposed in logs, image layers, environment files committed to Git, or version control.
Simple ga gurthupettuko: Docker lo secrets handle chesthunte → first passwords and API keys hardcode cheyyakudadhu → secrets image lo store cheyyakudadhu → secure secret management use cheyyi → runtime lo secrets provide cheyyi → logs and Git lo expose avvakunda chudu.
Eng: When handling Docker secrets → first avoid hardcoding passwords and API keys → do not store secrets in the image → use secure secret management → provide secrets at runtime → prevent exposure in logs and Git.
Example:
grep -R "password\|api_key\|secret" Dockerfile . → Dockerfile and source lo secrets hardcode ayyaya check cheyyi.
cat .gitignore → Secret files Git lo accidentally commit avvakunda ignore ayyaya verify cheyyi.
docker history myapp:latest → Image layers lo sensitive information accidentally unda check cheyyi.
docker secret ls → Docker Swarm environment lo configured secrets unnaya chudu.

Secret ni Dockerfile lo hardcode cheyyakunda → secure secret management use chesi → runtime lo provide chesi → logs and image lo expose avvakunda verify cheyyi.

---

21. Docker container is not coming back after server restart. What will you check?
Ans: First, I check whether the Docker service is running after the server restart. Then I check whether the container has an appropriate restart policy configured.
I also check the container status and startup logs to identify whether the container is failing during startup. If required, I correct the restart configuration and verify that the container comes back successfully after another restart.
Simple ga gurthupettuko: Server restart tarvatha container raakapothe → first Docker service running unda chudu → container restart policy check cheyyi → container status and startup logs verify cheyyi → startup failure unda identify cheyyi → restart configuration correct cheyyi → server restart tarvatha container malli vastunda verify cheyyi.
Eng: If the container does not come back after a server restart → first check the Docker service → verify the container restart policy → check container status and startup logs → identify startup failures → correct the restart configuration → verify the container comes back after a restart.
Example:
systemctl status docker → Server restart tarvatha Docker service running lo unda chudu.
docker ps -a → Container automatically start ayyinda check cheyyi.
docker inspect <container_id> → Restart policy configured unda verify cheyyi.
docker update --restart unless-stopped <container_id> → Required restart policy configure cheyyi.
docker restart <container_id> → Container manually restart chesi behavior verify cheyyi.
Server restart tarvatha → docker ps → Container automatic ga Running state lo vachinda verify cheyyi.